Functionality Guidelines

Navigation Components

Back Button

Back Button Functionality:

- **Navigation:**
  The back button should consistently return the user to the immediately previous screen or page.
- **History Management:**
  The back button should respect the user's history, allowing them to navigate through the same path as their actions.
- **State Management:**
  When applicable, the back button should also manage the state of the screen or element, returning it to its previous state.
- **Position:**
  The back button is to be placed on the top left corner of the screen.
- **Visual Cues:**
  A clear visual representation of the back icon (e.g., a left-facing arrow) is essential to make the button easily recognisable.
- **Context Awareness:**
  The back button should adapt to different contexts, dismissing modals or drawers if opened from within a flow, instead of navigating to a new screen.
- **Avoid Breaking Flow:**
  Pressing back should not unexpectedly exit the app or take the user to a home screen unless they've reached the root of the navigation.
- **Prevent Data Loss:**
  If navigating back could lead to unsaved data being lost, display a confirmation dialog drawer, popup or prompt the user to save or discard changes.
- **Accessibility Support:**
  Back buttons should be reachable using assistive technologies like screen readers, with accessible labels like "Go back to previous screen".
- **Platform Conventions:**
  a) Android: Back button behaviour should align with the system back gesture/ . hardware button.
  b) iOS: Prefer < Back in the top-left with text describing the previous screen's title when appropriate.
- **Don't Use for Cancel Actions:**
  Avoid using the back button to cancel a flow (like form submission or confirmation screens). Use a separate Cancel or Close (X) option instead.
- **Progressive Disclosure**
  In multi-step forms or onboarding flows, use the back button to step back one level rather than exit entirely, keeping progress intact.

Close Button

Close Button Functionality:

- **Window/Element Termination:**
  The close button should always completely close the window, overlay, or element it pertains to, returning the user to the previous state/view.
- **Contextual Dismissal:**
  The close icon should only dismiss the immediate context (modal, pop-up, drawer, sheet). It should never navigate through app history or act like a back button.
- **No Permanent Actions:**
  Avoid placing critical or irreversible actions behind a close icon. Users expect it to be safe and dismissive, not perform a task.
- **Data Loss Confirmation:**
  If a user is likely to lose data by closing a window, a confirmation dialog should be displayed to prevent accidental data loss.
- **Position:**
  The close button is often positioned in the top right corner of the window or element it controls.
- **Visual Cues:**
  A clear visual representation of the close icon (e.g., an "X" or a cross) is essential for easy recognition.
- **Dismiss vs. Cancel Clarity:**
  In some contexts (like forms), differentiate between 'Close' and 'Cancel':
  - Close = exit the view
  - Cancel = discard current changes
- Avoid overlapping these behaviours.

General Guidelines for BACK and CLOSE button

- **Tap Target Size:**
  Ensure the icon shas a minimum 40×40 px tap target area for accessibility and touch precision, even if the icon itself is visually smaller.
- **Consistency:**
  Maintain consistent placement and appearance of back and close buttons across the application to ensure predictability.
- **Meaningful Interactions:**
  The interactions of back and close buttons should be intuitive and predictable, following established design patterns.
- **User Feedback:**
  Provide clear visual feedback (e.g., animation) when a user clicks on a back or close button to confirm the action and enhance the user experience.